



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

HOSPITAL RESOURCE AND PATIENT MANAGEMENT SYSTEM WITH EMERGENCY BED ALLOCATION

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for
the Course of*

DSAO1O1 – Object Oriented Programming with C++

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

Submitted by

B BALASURIYA (1911260010)

**Under the Supervision of
Dr. K. Parimala Kanaga Devan**

**SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences Chennai-602105**

SEPTEMBER 2026

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

BONAFIDE CERTIFICATE

**This is to certify that the Capstone Project entitled “Hospital Resource and Patient Management System with Emergency Bed Allocation” has been carried out by,
B Balasuriya (1911260010) under the supervision of Dr. Parimala Kanaga Devan and is submitted in partial fulfilment of the requirements for the current semester of the Bachelor of Technology at Saveetha Institute of Medical and Technical Sciences, Chennai.**

COURSE COORDINATOR

**Dr. C. Sivasankar
Professor
Department of CSE
Saveetha School of Engineering
SIMATS, Chennai – 602105.**

COURSE FACULTY

**Dr. K. Parimala Kanaga Devan
Professor
Department of CSE,
SIMATS Engineering,
SIMATS, Chennai - 602105**

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I am B Balasuriya of SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled **“Hospital Resource and Patient Management System with Emergency Bed Allocation”** is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: September 2026

Signature of the Students with Names

B Balasuriya

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Project Title: Hospital Resource & Patient Management System

Student	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Balasuriya	Server/back-end logic (registration, bed allocation, discharge, audit logging)	C++ HospitalSystem class, file persistence (patients/beds/resources), HTTP server & socket handling	API endpoint testing and bed-allocation logic testing	Chapters 3 & 4	35%
	Front-end dashboard, bed map UI, patient registration form	HTML/CSS/JS dashboard, bed grid UI, dynamic tables and forms, resource module	UI testing across browsers, responsive layout checks	Chapters 1, 2 & 5	35%
	Resource & inventory management, admin report generation, integration testing	Resource CRUD module, JSON API for resources/reports, admin report text generation	Resource update validation, end-to-end system testing, report accuracy verification	Chapters 6, 7 & References	30%
Total					100%

ABSTRACT

Hospital emergency departments routinely face the problem of matching an unpredictable and often urgent inflow of patients against a fixed and scarce number of beds, monitoring equipment and consumables. When this matching is done manually on paper registers or spreadsheets, the process is slow, error-prone and difficult to audit, and it frequently results in avoidable delays for critical patients while lower-acuity patients occupy high-dependency beds. This project addresses that problem by designing and implementing a self-contained Hospital Resource and Patient Management System with automated, condition-based emergency bed allocation.

The system is implemented in native C++ as a single executable that embeds a lightweight multi-threaded HTTP server (built on Winsock2) and serves a browser-based dashboard, so that no external web server, database engine or internet connection is required. On registration, a receptionist or triage staff member records the patient's demographic details together with clinical observations (SpO₂, heart rate and presenting complaint) and selects a Condition category — CRITICAL, HIGH, MEDIUM or LOW. The selected Condition is the sole, deterministic input to the priority engine, which immediately attempts to allocate a bed from a pool of 50 beds partitioned into 10 ICU, 15 Emergency and 25 General beds, searching the wards in an order tailored to the patient's priority (for example, CRITICAL patients are matched against ICU first, then Emergency, then General). Bed occupancy, patient records and a shared resource inventory (ventilators, cardiac monitors, infusion pumps, oxygen cylinders and wheelchairs) are persisted to disk in a simple pipe-delimited flat-file format after every state-changing action, and every registration, allocation, discharge, bed-status change and resource update is written to a timestamped audit log.

A responsive single-page dashboard, delivered by the same executable, presents live bed-occupancy statistics, a colour-coded 50-bed grid with an interactive per-bed action menu, a sortable patient list ranked by priority, an editable resource inventory, and an on-demand textual administrative report. The system was verified through functional testing of the registration-to-allocation-to-discharge lifecycle, boundary testing of bed exhaustion and resource limits, and concurrency testing of simultaneous requests, and it consistently reproduced the intended clinical work-flow: higher-priority patients preferentially receive higher-dependency beds, discharged patients release their bed immediately, and the dashboard reflects the true server-side state within one polling cycle. The resulting prototype demonstrates that a rule-based, condition-driven allocation engine combined with a lightweight embedded server can deliver a dependable, auditable and easily deployable hospital resource management tool suitable for small and mid-sized facilities or for use as an emergency-preparedness overlay in larger hospitals.

Keywords

Hospital Management System; Emergency Bed Allocation; Triage Priority; Embedded HTTP Server; Resource Inventory Management; Patient Registration Workflow.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	4
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	6
4	CHAPTER 4: Implementation, Testing & Results, Project Management	12
5	CHAPTER 5: Conclusion, Future Work and Reflection	16
	References	18
	Appendices	19

Note: page numbers above are placeholders — update them (or regenerate the table) once the final pagination of this document is fixed.

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative analysis of existing hospital management approaches	5
3.1	Functional and non-functional requirements	6
3.2	Design alternatives evaluation matrix	7
3.3	Risk register	11
4.1	Test plan and verification results	13
4.2	Achievement of objectives	14

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
ICU	Intensive Care Unit	—
ER	Emergency Room / Ward	—
SpO ₂	Peripheral capillary oxygen saturation	%
HR	Heart Rate	BPM
HTTP	HyperText Transfer Protocol	—
API	Application Programming Interface	—
JSON	JavaScript Object Notation	—
GUI	Graphical User Interface	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Hospitals coordinate three scarce resources simultaneously during every admission: beds, clinical equipment, and staff attention. In many small and mid-sized facilities this coordination is still performed with paper registers, whiteboards or generic spreadsheets. The Hospital Resource and Patient Management System developed in this project is a self-contained, native C++ application that embeds its own HTTP server (using the Windows Sockets 2 API) and exposes a browser-based dashboard on localhost. It manages a fixed pool of 50 beds across ICU, Emergency and General wards, a shared inventory of critical-care equipment, and a patient registration and discharge workflow in which bed allocation is driven automatically by a clinically selected Condition category.

1.2 Need for the Project

Manual or spreadsheet-based bed tracking does not scale under emergency load: entries can be forgotten, a bed can be marked occupied and never released after discharge, and there is no single, always-current view of which wards have capacity. Front-desk and triage staff are the primary users affected, and secondarily hospital administrators who need situational awareness of capacity and equipment. Existing generic hospital information systems are often heavyweight, require database servers and network infrastructure, and are difficult to deploy quickly in a small clinic, a field hospital, or as a stand-alone emergency-overload tool alongside a larger Hospital Information System (HIS). The engineering significance of the project lies in showing that a dependable, auditable allocation engine can be delivered as a single portable executable with no external dependencies, while still enforcing a clear, deterministic clinical policy for who gets which bed.

1.3 Problem Statement

Design and implement a stand-alone hospital resource and patient management system that (a) accepts patient registration together with clinical observations, (b) deterministically converts a selected patient Condition into a priority level and an emergency bed allocation across a constrained multi-ward bed pool, (c) keeps bed, patient and equipment state consistent and persistent across restarts, and (d) exposes this functionality through a concurrent, multi-client web interface — while resolving the conflicting requirements of allocation speed (a bed must be found in milliseconds during an emergency), allocation fairness (higher-acuity patients should not be blocked behind lower-acuity ones), data integrity (bed and patient state must never diverge, e.g. an occupied bed must always correspond to an active patient), and deployability (no database server, no internet connection, no installation step beyond running the executable).

1.4 Project Aim

To design, implement and evaluate a lightweight, self-hosted hospital resource and patient management system that automates emergency bed allocation using a condition-based priority rule and provides real-time visibility of bed occupancy, patient status and critical-care resources through a web dashboard.

1.5 Project Objectives

- To design a data model for patients, beds and resources that supports condition-based priority and ward-preference allocation.
- To develop a thread-safe C++ back end that performs registration, automatic bed allocation, discharge and resource updates while keeping all state internally consistent.
- To implement a flat-file persistence layer and an append-only audit log so that the system survives restarts and every state change is traceable.
- To implement an embedded, multi-threaded HTTP server exposing a JSON/REST-style API, removing the need for a separate web server.
- To design and build a responsive browser dashboard for registration, patient tracking, a visual 50-bed grid, resource management and administrative reporting.
- To test the system's allocation logic, concurrency behaviour and data-consistency guarantees under normal and boundary conditions (e.g. bed exhaustion), and to evaluate the achievement of the stated objectives.

1.6 Scope

Included: patient registration with clinical detail capture, condition-based priority assignment, automated bed allocation and manual bed-status control (Available / Reserved / Maintenance), a fixed 50-bed pool (10 ICU, 15 Emergency, 25 General), a resource inventory module for five categories of equipment, a live dashboard, and a plain-text administrative report.

Excluded: user authentication and role-based access control, integration with external Hospital Information Systems or insurance/billing systems, encrypted network transport (the server binds to loopback only), mobile-native applications, and machine-learning-based triage.

Assumptions: the application runs on a single Windows host reachable at localhost:8080; the number and layout of beds is fixed at design time (50 beds); staff select the Condition category responsibly, since it is the sole determinant of priority.

Boundary conditions: when no bed of any ward is available, the patient is registered with status “Waiting” and no bed is assigned (allocatedBed = NO_BED); resource quantities are constrained so that available can never exceed total.

1.7 Expected Engineering Outcome

The expected outcome is a working software prototype — a single portable Windows executable — that functions as an emergency-department resource and patient management tool. Concretely, the deliverable is: (i) a C++ back-end system (HospitalSystem class) encapsulating patients, beds and resources with file-backed persistence and audit logging; (ii) an embedded multi-threaded HTTP server exposing REST-style endpoints for dashboard data, registration, discharge, bed-status changes and resource updates; and (iii) a browser-rendered single-page dashboard giving hospital staff real-time visibility and control over bed occupancy, patient priority and equipment availability.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant approaches were identified by examining (a) commercially available Hospital Information System (HIS) and Bed Management System (BMS) product literature, (b) open-source hospital/clinic management projects, and (c) established triage frameworks used in emergency medicine (e.g. category-based triage scales), to understand how bed allocation and patient prioritisation are typically automated and what trade-offs existing tools make between features, cost and deployability.

2.2 Review of Existing Work

Commercial Hospital Information Systems typically provide comprehensive admission, discharge and transfer (ADT) modules integrated with electronic health records, billing and pharmacy systems. These systems offer rich functionality but require dedicated database servers, network infrastructure, licensing and IT staff to deploy and maintain, which makes them impractical for small clinics, temporary field facilities or as a rapid emergency-overflow tool.

Open-source clinic/hospital management projects (commonly built on a web framework with a relational database such as MySQL or PostgreSQL) reduce licensing cost but still require a database server, a web server such as Apache or Nginx, and a hosting environment to be installed and configured before use, introducing a non-trivial deployment barrier in resource-constrained settings.

Clinical triage frameworks (for example scales that group patients into acuity levels) provide a standardized vocabulary for urgency but are procedural guidelines for humans; they do not, by themselves, specify how a computer system should map an assigned acuity level onto a concrete, limited pool of physical beds across multiple wards.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Commercial HIS with ADT module	Feature-rich, integrates billing/EHR	High cost, requires DB server, complex deployment	Motivates a lightweight alternative for small facilities
Open-source web-based clinic system	Free, customisable	Needs external DB + web server, setup overhead	Justifies embedding the server and using file persistence
Manual / spreadsheet tracking	No software cost, familiar	Error-prone, no automatic allocation, stale data	Baseline against which automation benefits are measured
Clinical triage scale (acuity levels)	Standardised clinical vocabulary	Does not map acuity to a concrete bed pool	Adopted conceptually as the Condition-to-priority mapping

2.4 Research / Engineering Gap

None of the reviewed approaches combine (i) a zero-external-dependency deployment model, (ii) an automatic, deterministic mapping from a clinically selected condition to a ward-ordered bed search, and (iii) a built-in, always-on web dashboard, in a single portable artefact. Existing systems either require external infrastructure (HIS, open-source web stacks) or provide no computational allocation logic at all (manual tracking, triage scales alone).

2.5 Proposed Contribution

This project contributes a self-contained C++ application that removes the external-infrastructure barrier by embedding its own multi-threaded HTTP server and using a simple flat-file persistence layer instead of a database engine, while still implementing a deterministic, auditable, condition-based bed-allocation policy and a full browser dashboard — a combination not offered by the reviewed commercial, open-source or purely procedural approaches.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Triage / front-desk staff	Register a patient with clinical details and get an immediate bed allocation
Ward nurses	See at a glance which beds are occupied, available, reserved or under maintenance
Hospital administrator	View aggregate occupancy, priority mix and resource stock, and generate a report on demand
Biomedical / equipment in-charge	Track total vs. available quantities of critical equipment

Functional & Non-Functional Requirements

Requirement	Type (Functional / Non-Functional)	Measurable Target
Register patient and auto-allocate bed	Functional	Bed assigned or waiting status returned in a single request
Discharge patient and release bed	Functional	Bed status returns to Available within the same request
Manual bed status change	Functional	Reject the change if bed is Occupied and target is not Available
Resource inventory update	Functional	Reject update if available > total or either is negative
Concurrent client access	Non-Functional	Each client handled on its own thread; shared state protected by a mutex
Data durability	Non-Functional	Patients, beds and resources persisted to disk after every mutation
Deployability	Non-Functional	Single executable, no external DB / web server required

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
RFC 7230 (HTTP/1.1 message syntax)	Well-formed request/response framing	readRequest()/response() implement request-line, header and Content-Length handling
RFC 3986 (URI generic syntax) – form encoding conventions	application/x-www-form-urlencoded decoding	urlDecode()/parseForm() implement percent-decoding and '+' as space
JSON (ECMA-404) data interchange	Well-formed JSON responses	jsonEscape() and hand-built JSON serialisers in dashboardJson()/patientsJson()/etc.

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Total bed capacity	50 (10 ICU + 15 Emergency + 25 General)	beds	High
Allocation response time	Single synchronous call, no queuing delay	ms	High
Persistence write	After every state-changing action	—	High
Concurrent clients	One thread per accepted connection	clients	Medium
Resource categories tracked	5 (Ventilator, Cardiac Monitor, Infusion Pump, Oxygen Cylinder, Wheelchair)	types	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: A database-backed web application (e.g. Node.js/PHP with MySQL) served over a conventional web server, offering rich querying but requiring installation and configuration of a DB engine and web server.

Alternative 2: A desktop-only GUI application (e.g. built with a native Windows GUI toolkit) with no network interface, avoiding networking complexity but limiting access to a single machine and a single concurrent user.

Alternative 3 (selected): A single C++ executable embedding its own multi-threaded HTTP server and using flat-file persistence, combining zero-dependency deployment with multi-client, browser-based access.

Criterion	Weight	Alternative 1 (DB + web server)	Alternative 2 (Desktop GUI)	Alternative 3 (Embedded server)
Performance	20%	High	High	High
Cost / Deployability	30%	Low (needs DB/server)	Medium	High
Safety (data consistency)	20%	High (ACID DB)	Medium	Medium–High (mutex-guarded, file-backed)
Sustainability (maintainability)	15%	Medium	Medium	High (single codebase, no infra)
Reliability	15%	High	Medium	High

3.5 Selected Design & Detailed Design

Alternative 3 was selected because it best satisfies the dominant weighted criterion — cost and deployability — while still meeting performance and reliability needs through in-memory data structures guarded by a single mutex and disciplined disk persistence, evidenced directly in the accompanying source (HospitalSystem::registerPatient, dischargePatient, setBedStatus and updateResource each take lock_guard<mutex> before mutating shared state and immediately call the corresponding saveXUnlocked() routine).

System architecture: A single process hosts three cooperating layers — (1) a HospitalSystem domain layer holding vectors of Patient, Bed and Resource objects behind a mutex, with load/save routines for three pipe-delimited flat files (patients.txt, beds.txt, resources.txt) and an append-only audit.log; (2) an HttpServer layer built directly on Winsock2 that accepts TCP connections on loopback port 8080, spawns one detached thread per client, parses the HTTP request line, headers and body, and routes GET/POST paths (/ , /api/dashboard, /api/patients, /api/beds, /api/resources, /api/report, /api/register, /api/discharge, /api/bed-status, /api/resource) to HospitalSystem methods; and (3) a single embedded HTML/CSS/JavaScript page (HTML_PAGE) served at '/', which polls the JSON endpoints every 5 seconds and renders the dashboard, registration form, patient table, bed grid and resource table entirely client-side.

Key engineering logic — priority-to-ward search order: on registration, the selected Condition (CRITICAL/HIGH/MEDIUM/LOW) is copied verbatim into both the patient's condition and priority fields, and a ward-search order is chosen accordingly: CRITICAL searches {ICU, Emergency, General}; HIGH searches {Emergency, ICU, General}; MEDIUM and LOW search {General, Emergency, ICU}. The first Available bed found in that order is marked Occupied, linked to the patient, and the patient's status becomes 'Admitted'; if no bed is found in any ward the patient remains 'Waiting' with bedId empty.

Systems approach: the three subsystems interface only through well-defined C++ method calls and a small JSON/plain-text wire protocol, so the persistence layer can be swapped (e.g. for a

real database) or the transport layer replaced (e.g. with a TLS-capable server) without touching the allocation logic, which is isolated inside registerPatient()).

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Flat-file persistence (pipe-delimited)	Embedded SQLite / full RDBMS	Zero external dependency, human-readable files	No transactional rollback, weaker query power	Flat files chosen: sufficient at this scale, and mutex-guarded saves keep state consistent
One thread per connection	Thread pool / async I/O (IOCP)	Simple to implement and reason about	Higher overhead under very high concurrency	Thread-per-connection chosen: acceptable at expected clinic-scale traffic
Condition dropdown as sole priority source	Auto-derive priority from SpO ₂ /HR thresholds	Deterministic, auditable, avoids brittle vital-sign heuristics	Relies on correct clinical judgement at data entry	Condition-only rule kept as an explicit design choice, clearly surfaced in the UI notice text
Loopback-only binding (127.0.0.1)	Bind to all interfaces for LAN access	Eliminates unauthenticated network exposure	Cannot be used multi-workstation without further hardening	Loopback binding retained for this prototype; flagged as future work

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Resource efficiency of the runtime	A native compiled C++ executable with in-memory vectors and flat-file I/O was compared against a database-backed stack requiring a persistently running DB server process	Chose the embedded, no-idle-service design to minimise continuous energy/CPU draw when the system is not in active use
Ethics	Fairness of the priority rule	Reviewed that the ward-search order in registerPatient() consistently favours CRITICAL patients for ICU beds before any other condition can claim them	Retained condition-only, code-visible priority logic (no hidden weighting) so the allocation rule can be audited and explained to clinical staff
Health & Safety	Risk of stale bed data misleading staff	Traced the discharge path: dischargePatient() clears bedId, sets status to 'Discharged', releases the bed via oldBed match and a fallback scan of b.patientId, and the dashboard forces cache:'no-store' GETs	Kept the fallback bed-release scan and no-cache fetches specifically to prevent a bed showing 'Occupied' after its patient has left
Security	Exposure of an unauthenticated HTTP service	Reviewed that the server binds to INADDR_LOOPBACK and sets Access-Control-Allow-Origin: * with no authentication layer	Accepted loopback-only exposure for this prototype; documented as a required addition (authentication/TLS) before any multi-user or networked deployment

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Bed/patient data left inconsistent after crash mid-write	Low	High	Medium	Save routines write the whole file (ios::trunc) atomically per call; audit log records every action for reconciliation	Small window during a crash mid-write remains
Concurrent requests corrupting shared vectors	Medium	High	Medium	All mutating methods take lock_guard<mutex> before touching patients/beds/resources	None observed under tested load
All 50 beds exhausted during a surge	Medium	Medium	Medium	Patient is registered as 'Waiting' rather than rejected; dashboard surfaces waiting count	Genuine bed shortage is a real-world constraint, not fully solvable in software
Unauthenticated access if firewall/port-forwarding misconfigured	Low	High	Medium	Server binds to loopback only by design	Depends on host OS network configuration remaining unchanged

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed incrementally: the domain classes (Patient, Bed, Resource) and the HospitalSystem logic were built and unit-tested first using console-level calls, then the Winsock2-based HttpServer was layered on top to expose that logic over HTTP, and finally the single-page dashboard was written against the resulting JSON API. Materials and resources used were entirely software-based: the Windows SDK (Winsock2/ws2tcpip), the C++ Standard Library, and a browser for the client, with no physical hardware, dataset or fabrication involved beyond the developer's workstation.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

Back end: written in ISO C++ using the Standard Library (<vector>, <map>, <mutex>, <thread>, <fstream>, <sstream>) for domain logic, and Winsock2 for a raw sockets-based HTTP/1.1 server that parses the request line, headers and Content-Length-delimited body, and routes ten distinct endpoints. Data model classes Patient, Bed and Resource are plain structs-with-methods managed inside HospitalSystem, which also owns three flat text files and one audit log.

Front end: a single embedded HTML document (HTML_PAGE) containing CSS for a card/table/bed-grid layout and vanilla JavaScript that fetches the JSON endpoints, renders the dashboard cards, the sortable patient table, the colour-coded 50-bed grid with a click-to-open bed action menu, and the resource inventory editor, polling the server every 5 seconds via setInterval so occupancy and priority counts stay current.

Tool / Software / Equipment	Purpose	Stage Used
Microsoft C++ compiler / Windows SDK	Compile the executable and provide Winsock2 headers	Implementation
Winsock2 (ws2tcpip.h)	TCP socket server, request/response I/O	Implementation
Browser DevTools	Inspect fetch requests/responses to the JSON API	Testing
Plain-text flat files (patients.txt, beds.txt, resources.txt) + audit.log	Persistence and traceability	Implementation & Testing

4.3 Test Plan & Verification

The test plan was defined before results were collected, covering the registration-to-allocation lifecycle, discharge/bed-release, boundary conditions (bed exhaustion, resource limits) and concurrency.

Requirement	Test Method	Acceptance Criterion
CRITICAL patient preferentially gets an ICU bed	Register a CRITICAL patient while ICU beds remain available	Allocated bed id begins with 'ICU-'
Patient is queued when no bed matches	Fill all 50 beds, then register a new patient	Response is OK id priority NO_BED and patient status is 'Waiting'
Discharge releases the correct bed	Discharge an admitted patient, then re-fetch /api/beds	Bed status returns to 'Available', patientId resets to 0
Bed-status change is rejected on occupied beds	POST /api/bed-status with an Occupied bed and status=Maintenance	Response begins with 'ERROR '
Resource update rejects invalid quantities	POST /api/resource with available > total	Response begins with 'ERROR '
Dashboard reflects true state after discharge	Discharge via the bed-menu, then reload the Beds tab	Bed renders green ('available') without manual page refresh

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Total bed pool	50 beds (10/15/25)	50 beds (10/15/25)	Pass
CRITICAL → ICU-first allocation	ICU bed chosen when free	ICU bed chosen when free	Pass
Waiting-state on exhaustion	Status = Waiting, bed = NO_BED	Status = Waiting, bed = NO_BED	Pass
Bed release on discharge	Bed status = Available	Bed status = Available	Pass
Rejection of invalid resource update	ERROR response	ERROR response	Pass

4.4 Results

Across the executed test scenarios, all planned acceptance criteria in Table 4.2 were met. Registering patients in decreasing order of priority (CRITICAL, HIGH, MEDIUM, LOW) while beds were still available consistently produced ICU, Emergency, General and General bed allocations respectively, matching the ward-search order encoded in registerPatient(). Filling all 50 beds and then registering an additional patient of any priority correctly returned a 'Waiting' status with allocatedBed = NO_BED, and the dashboard's waiting-patient counter incremented accordingly. Discharging a patient through both the patient-table action and the bed-grid action menu correctly released the bed in every case, including a deliberately corrupted case where the patient's stored bedId was cleared but the bed still pointed at the patient — the fallback release scan in dischargePatient() correctly located and released that bed.

4.5 Data Analysis & Engineering Judgment

The condition-only priority rule performed exactly as specified in every trial: because priority is copied directly from the selected Condition string with no secondary weighting, there was no observed case of priority drift or inconsistent ward preference. The one qualitative limitation observed is that the rule is only as trustworthy as the triage judgement of the person selecting the Condition; SpO₂ and heart rate are captured and stored (and visible to staff) but do not automatically override an incorrect Condition selection. This is treated as an accepted engineering trade-off (see Table 3.2, row 3) rather than a defect, since automatic vital-sign-based overriding would introduce a second, less transparent allocation rule that could itself be wrong or exploited.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Design a data model supporting condition-based priority and ward preference	Patient/Bed/Resource classes and preferredWard()/ward-order logic in registerPatient()	Fully Achieved
Develop a thread-safe C++ back end	mutex-guarded HospitalSystem methods (lock_guard<mutex> in every mutator)	Fully Achieved
Implement persistence and audit logging	patients.txt/beds.txt/resources.txt + audit.log with auditUnlocked() calls on every action	Fully Achieved
Implement an embedded multi-threaded HTTP server	Winsock2 HttpServer with per-connection detached threads and ten routed endpoints	Fully Achieved
Build a responsive dashboard	HTML_PAGE with dashboard cards, bed grid, patient table, resource editor and report viewer	Fully Achieved
Test allocation logic, concurrency and consistency	Test plan in Section 4.3 executed against registration, discharge, boundary and concurrent scenarios	Fully Achieved

Compared with the reviewed commercial and open-source solutions (Chapter 2), the delivered system trades away integrated billing/EHR features and multi-machine concurrent database access, but gains near-zero deployment overhead and a fully transparent, code-visible allocation rule.

4.7 Strengths & Limitations

Strengths:

- Zero external dependencies — a single executable serves both the API and the UI.
- Deterministic, auditable priority rule with a full audit trail of every registration, allocation, discharge, bed-status change and resource update.
- Automatic and manual bed-state safeguards (occupied beds cannot be silently reassigned; discharge has a fallback release path for inconsistent data).
- Real-time-feeling dashboard via short polling intervals and no-cache GET requests.

Limitations:

- No authentication or role-based access control; anyone able to reach the loopback port can register, discharge or reconfigure beds.
- Bed pool size and ward split (10/15/25) are compiled in rather than configurable at runtime.
- Flat-file persistence lacks transactional guarantees compared with a full RDBMS.
- The system currently binds to loopback only, limiting it to single-machine use without further networking/security work.

4.8 Project Planning, Roles & Budget

Milestone Timeline

Milestone	Planned Duration	Status
Requirements gathering & data-model design	Week 1–2	Completed
Domain logic (HospitalSystem) implementation	Week 3–4	Completed
Embedded HTTP server implementation	Week 5–6	Completed
Dashboard (HTML/CSS/JS) implementation	Week 6–7	Completed
Testing, verification and report writing	Week 8–9	Completed

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to remove the operational fragility of manual bed tracking in an emergency-care setting by building a self-contained Hospital Resource and Patient Management System with automatic, condition-based emergency bed allocation. The delivered system implements a thread-safe, file-persisted domain model of 50 beds across ICU, Emergency and General wards, a deterministic priority engine that maps a clinically selected Condition onto a ward-ordered bed search, an embedded multi-threaded HTTP server requiring no external database or web server, and a real-time browser dashboard for registration, patient tracking, bed management and reporting. Verification testing confirmed that the allocation, discharge and resource-update logic behaves correctly under normal operation, boundary conditions such as bed exhaustion, and simple concurrency scenarios, meeting all six stated project objectives. The project demonstrates that a lightweight, dependency-free engineering approach can deliver a dependable and auditable hospital resource management tool suitable for small facilities or as a rapid-deployment emergency overlay.

5.2 Future Work

- Add authentication and role-based access control before any deployment beyond a trusted, single-user environment.
- Make ward sizes and the total bed count configurable, and support multiple hospital sites from one instance.
- Replace loopback-only binding with a properly secured (TLS) LAN/WAN-capable listener for genuine multi-workstation use.
- Migrate flat-file persistence to an embedded transactional store (e.g. SQLite) for stronger durability guarantees.
- Add historical analytics (occupancy trends, average length of stay) on top of the existing audit log.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical use of the Windows Sockets 2 API to build an HTTP server from raw TCP sockets.
- Designing thread-safe shared state in C++ using `std::mutex` and `lock_guard`.
- Structuring a small, dependency-free full-stack application (native back end + embedded web front end).

Engineering & Professional Skills Developed:

- Requirements analysis and translating a clinical workflow (triage → allocation → discharge) into deterministic software logic.
- Systematic verification/testing planning before results collection.
- Documentation and audit-trail design for accountability in a safety-relevant domain.

Individual Reflection (each student, 4–5 lines):

- Most difficult engineering problem encountered and how it was solved: [To be completed by each student]
- A key engineering decision made personally and the evidence behind it: [To be completed by each student]
- New knowledge acquired independently: [To be completed by each student]
- What would be redesigned if repeated: [To be completed by each student]

REFERENCES

A consistent recognised referencing style such as IEEE has been used. All literature, standards, websites, datasets, software, open-source resources and AI-assisted resources used are acknowledged below.

[1] Internet Engineering Task Force, “Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing,” RFC 7230, 2014.

[2] Internet Engineering Task Force, “Uniform Resource Identifier (URI): Generic Syntax,” RFC 3986, 2005.

[3] Microsoft Corporation, “Winsock2 Reference,” Microsoft Learn documentation, docs.microsoft.com.

[4] ISO/IEC 14882, “Programming Languages — C++,” International Organization for Standardization.

[5] Anthropic, “Claude,” AI-assisted resource used for report drafting and structuring, claude.ai.

APPENDICES

Appendix A – Detailed Calculations

Not applicable — the system is rule-based rather than numerically computational; the only arithmetic performed is bed/resource counting, which is shown directly in Chapter 4 results.

Appendix B – Engineering Drawings / Schematics

See Figure 1.1 (system block diagram) and Figure 3.1 (bed allocation decision flow) referenced in the List of Figures.

Appendix C – Source Code / Code Repository Information

The complete source (HospitalSystem, HttpServer and embedded dashboard) is maintained as a single C++ file, HOSPITAL_CODE.txt, in the project repository. Key excerpt — the priority-to-ward-order rule from registerPatient():

```
if (p.priority == "CRITICAL") order = {"ICU","Emergency","General"}; else if  
(p.priority == "HIGH") order = {"Emergency","ICU","General"}; else order =  
{"General","Emergency","ICU"};
```

Appendix D – Datasheets

Not applicable — no external hardware components were used in this software-only project.

Appendix E – Experimental / Raw Data

Raw test artefacts are the generated patients.txt, beds.txt, resources.txt and audit.log files produced during the test scenarios described in Section 4.3.

Appendix F – Ethics / Institutional Approval

Not applicable — the project uses only synthetic test data and does not process real patient records.

Appendix G – Project Meeting / Progress Records

[Insert meeting minutes / progress log here.]

Appendix H – User/Industry Feedback

[Insert user or industry feedback here, if collected.]

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable at the time of this report.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement table earlier in this report; supporting evidence (commit history, task logs) to be attached separately.